МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ КАЗАХСТАН
«НАО Евразийский национальный университет имени Л. Н. Гумилёва»
Факультет информационных технологий
Кафедра Информационная безопасность

ОТЧЕТ

по практической работе № 5

Дисциплина: «Клиент-серверные технологии и их безопасность»

Тема: «Реализация ролевой модели разграничения доступа (RBAC)»
Тема мини-проекта: «CI/CD Security Pipeline: настройка автоматического сканирования уязвимостей исходного кода (SAST) и зависимостей (SCA) в GitHub Actions / GitLab CI»

Руководитель, преподаватель _____ Казиева Н.М.
«___» _____ 2026

Выполнил магистрант группы МСИБ-24
Нуртазинов Ислам Тулегенович
«___» _____ 2026

Астана, 2026

1. Цель и задачи работы

Цель работы — реализовать ролевую модель разграничения доступа (RBAC) в клиент-серверном приложении и настроить для него пайплайн CI/CD, который при каждом изменении кода автоматически проверяет исходный код (SAST) и зависимости (SCA) на уязвимости.

Задачи:

- спроектировать базу данных RBAC: пользователи, роли, разрешения и связи между ними;
- реализовать REST API на FastAPI с тремя ресурсами разного уровня доступа;
- настроить в GitHub Actions автоматический запуск SAST, SCA и тестов ролевой модели, подготовить такой же пайплайн для GitLab CI;
- показать работу пайплайна на версии с уязвимостями и на исправленной версии;
- провести негативное тестирование ролевой модели (тесты 1–3, IDOR, эскалация привилегий).

2. Краткие теоретические сведения

RBAC (Role-Based Access Control) — модель, в которой права назначаются не пользователям напрямую, а ролям. Пользователь получает одну или несколько ролей, роль объединяет набор разрешений, а разрешение описывает действие над ресурсом. Чтобы изменить права сотрудника, достаточно сменить его роль. Модель опирается на принцип наименьших привилегий (PoLP): каждой роли выдаются только те разрешения, которые нужны для её задач.

SAST (Static Application Security Testing) — анализ исходного кода без его запуска. Инструмент ищет в коде опасные конструкции: SQL-запросы, собранные из строк, слабые алгоритмы хеширования, секреты в тексте программы. В работе использован Bandit — анализатор для Python.

SCA (Software Composition Analysis) — проверка сторонних библиотек. Инструмент сравнивает версии зависимостей с базами известных уязвимостей. В работе использован pip-audit, который проверяет файл requirements.txt по базам PyPI Advisory Database и OSV, включая транзитивные зависимости.

Если эти проверки встроены в пайплайн CI/CD, они выполняются при каждом push и pull request. Уязвимость обнаруживается до слияния изменения в основную ветку, а не после выпуска (подход «shift-left»).

Таблица 1 — Сравнение SAST и SCA

	SAST	**SCA**
Что проверяется	Собственный исходный код	Сторонние библиотеки (зависимости)
Что находит	Ошибки в коде: инъекции, слабая криптография, секреты	Известные уязвимости (CVE) в используемых версиях
Инструмент в работе	Bandit	pip-audit
Способ исправления	Изменить код	Обновить версию библиотеки

3. Проектирование базы данных RBAC

База данных состоит из пяти таблиц. Таблицы users, roles и permissions хранят пользователей, роли и разрешения. Таблицы user_roles и role_permissions реализуют связи «многие-ко-многим». Внешние ключи объявлены с ON DELETE CASCADE и ON UPDATE CASCADE: при удалении пользователя или роли связанные записи удаляются автоматически.

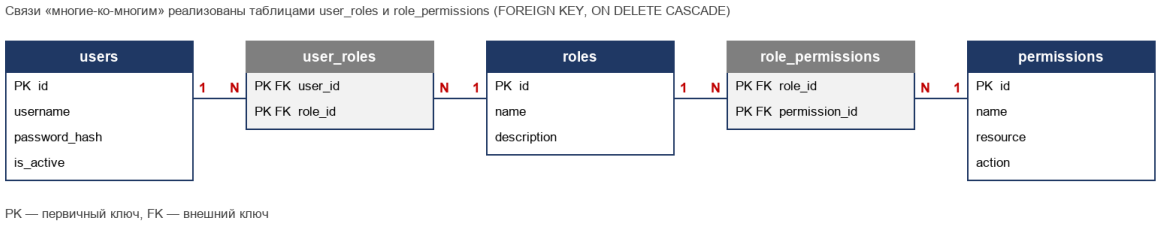


Рисунок 1 — ER-диаграмма базы данных RBAC

```
CREATE TABLE users (
    id            SERIAL PRIMARY KEY,
    username      VARCHAR(50)  NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    is_active     BOOLEAN      NOT NULL DEFAULT TRUE
);

CREATE TABLE roles (
    id          SERIAL PRIMARY KEY,
    name        VARCHAR(50) NOT NULL UNIQUE,
    description TEXT
);

CREATE TABLE permissions (
    id       SERIAL PRIMARY KEY,
    name     VARCHAR(50) NOT NULL UNIQUE,
    resource VARCHAR(50) NOT NULL,
    action   VARCHAR(20) NOT NULL
);

-- Связь многие-ко-многим: пользователь <-> роль
CREATE TABLE user_roles (
    user_id INTEGER NOT NULL REFERENCES users(id) ON DELETE CASCADE ON UPDATE CASCADE,
    role_id INTEGER NOT NULL REFERENCES roles(id) ON DELETE CASCADE ON UPDATE CASCADE,
    PRIMARY KEY (user_id, role_id)
);
```

```
-- Связь многие-ко-многим: роль <-> разрешение
CREATE TABLE role_permissions (
    role_id       INTEGER NOT NULL REFERENCES roles(id)       ON DELETE CASCADE ON UPDATE
CASCADE,
    permission_id INTEGER NOT NULL REFERENCES permissions(id) ON DELETE CASCADE ON UPDATE
CASCADE,
    PRIMARY KEY (role_id, permission_id)
);
```

Листинг 1 — SQL-скрипт создания таблиц (sql/rbac_schema.sql, PostgreSQL)

Демонстрационный сервис использует ту же схему в SQLite, чтобы запускаться без установки СУБД. Скрипт для PostgreSQL находится в файле sql/rbac_schema.sql.

Таблица 2 — Матрица «роль — разрешения»

Роль	**VIEW_PUBLIC**	**VIEW_AUDIT**	**MANAGE_USERS**
USER	да	—	—
MODERATOR	да	да	—
ADMIN	да	да	да

4. Реализация сервиса и проверки прав

Сервис написан на Python (FastAPI) и состоит из трёх файлов: app/database.py — работа с БД, app/auth.py — аутентификация и проверка прав, app/main.py — эндпоинты. После входа пользователь получает JWT-токен, в котором записан только его идентификатор. Роли и разрешения при каждом запросе читаются из базы данных.

Таблица 3 — Эндпоинты сервиса

Метод и путь	**Кому доступно**	**Проверка**
POST /api/v1/login	Всем	Логин и пароль
GET /api/v1/public	Всем авторизованным	Действительный токен
GET /api/v1/moderator/audit	MODERATOR, ADMIN	require_role
GET, POST /api/v1/admin/users	ADMIN с правом MANAGE_USERS	require_role + check_permission
PUT /api/v1/admin/users/{id}/role	ADMIN с правом MANAGE_USERS	require_role + check_permission
GET /api/v1/users/{id}	Владелец профиля или MANAGE_USERS	Сравнение id с id из токена

Проверка прав реализована как зависимости FastAPI (аналог middleware). Функция get_current_user проверяет токен и возвращает ошибку 401, если он отсутствует или недействителен. Функции require_role и check_permission читают роли и разрешения пользователя из БД и возвращают 403, если прав недостаточно.

```
def get_current_user(cred: HTTPAuthorizationCredentials | None = Depends(bearer)):
```

```
    """Аутентификация: кто делает запрос. Без действительного токена — 401."""
    if cred is None:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Требуется токен")
    try:
        payload = jwt.decode(cred.credentials, SECRET_KEY, algorithms=["HS256"])
    except jwt.PyJWTError:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Недействительный токен")
    user = database.get_user(int(payload["sub"]))
    if user is None or not user["is_active"]:
        raise HTTPException(status.HTTP_401_UNAUTHORIZED, "Учётная запись неактивна")
    return {"id": user["id"], "username": user["username"]}


def require_role(*allowed_roles):
    """Авторизация по роли: у пользователя должна быть хотя бы одна из ролей."""
    def dependency(current_user=Depends(get_current_user)):
        user_roles = database.get_roles(current_user["id"])
        if not user_roles & set(allowed_roles):
            raise HTTPException(status.HTTP_403_FORBIDDEN,
                                "Недостаточно прав для выполнения операции")
        return current_user
    return dependency


def check_permission(required_permission):
    """Авторизация по разрешению: оно должно быть у одной из ролей пользователя."""
    def dependency(current_user=Depends(get_current_user)):
        user_permissions = database.get_permissions(current_user["id"])
        if required_permission not in user_permissions:
            raise HTTPException(status.HTTP_403_FORBIDDEN,
                                "Недостаточно прав для выполнения операции")
        return current_user
    return dependency
```

Листинг 2 — Проверка токена, ролей и разрешений (app/auth.py)

```
# 2. Доступно только ролям MODERATOR и ADMIN
@app.get("/api/v1/moderator/audit")
def audit(current_user=Depends(require_role("MODERATOR", "ADMIN"))):
    return {"requested_by": current_user["username"], "users": database.list_users()}


# 3. Доступно только роли ADMIN с правом MANAGE_USERS
@app.get("/api/v1/admin/users", dependencies=[Depends(require_role("ADMIN"))])
def list_users(current_user=Depends(check_permission("MANAGE_USERS"))):
    return database.list_users()


@app.post("/api/v1/admin/users", status_code=201,
dependencies=[Depends(require_role("ADMIN"))])
def create_user(body: UserIn, current_user=Depends(check_permission("MANAGE_USERS"))):
    try:
        user_id = database.create_user(body.username, hash_password(body.password), body.role)
    except sqlite3.IntegrityError:
        raise HTTPException(status.HTTP_409_CONFLICT, "Пользователь уже существует")
    return {"id": user_id, "username": body.username, "role": body.role}


# Защита от IDOR: чужой профиль виден только обладателю права MANAGE_USERS
@app.get("/api/v1/users/{user_id}")
def profile(user_id: int, current_user=Depends(get_current_user)):
    is_owner = user_id == current_user["id"]
    if not is_owner and "MANAGE_USERS" not in database.get_permissions(current_user["id"]):
        raise HTTPException(status.HTTP_403_FORBIDDEN, "Недостаточно прав для выполнения
```

```
операции")
    user = database.get_user(user_id)
    if user is None:
        raise HTTPException(status.HTTP_404_NOT_FOUND, "Пользователь не найден")
    return {"id": user["id"], "username": user["username"], "roles":
sorted(database.get_roles(user_id))}
```

Листинг 3 — Защищённые эндпоинты (app/main.py)

5. Пайплайн безопасности CI/CD

Пайплайн описан в файле .github/workflows/security.yml и запускается при каждом push в ветку main и при каждом pull request. Он состоит из трёх независимых заданий, которые выполняются параллельно.

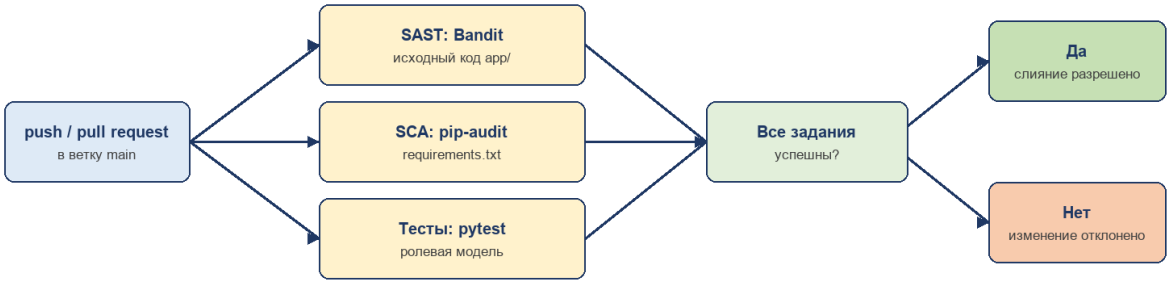


Рисунок 2 — Схема пайплайна безопасности

- SAST (Bandit): команда bandit -r app -ll проверяет каталог app. Задание завершается ошибкой, если найдены проблемы уровня MEDIUM или HIGH.
- SCA (pip-audit): команда pip-audit -r requirements.txt проверяет зависимости. Задание завершается ошибкой, если хотя бы у одной библиотеки есть известная уязвимость.
- Тесты RBAC (pytest): 8 тестов проверяют, что каждая роль получает доступ только к своим ресурсам.

Если хотя бы одно задание завершилось ошибкой, запуск помечается как неуспешный. При включённой защите ветки (Settings → Branches → Require status checks to pass) такое изменение нельзя слить в main.

```
name: Security Pipeline

# Пайплайн запускается при каждом push в main и при каждом pull request
on:
  push:
    branches: [main]
  pull_request:

permissions:
  contents: read

jobs:
  # SAST: статический анализ исходного кода
  sast:
```

```
    name: SAST (Bandit)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v7
        with:
          python-version: "3.12"
      - run: pip install -r requirements-dev.txt
      # -ll: задание падает, если есть находки уровня MEDIUM и выше
      - run: bandit -r app -ll

  # SCA: проверка зависимостей по базе известных уязвимостей
  sca:
    name: SCA (pip-audit)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v7
        with:
          python-version: "3.12"
      - run: pip install -r requirements-dev.txt
      # задание падает, если хотя бы у одной зависимости есть известная уязвимость
      - run: pip-audit -r requirements.txt

  # Тесты ролевой модели: права доступа не должны сломаться при изменениях
  tests:
    name: Тесты RBAC (pytest)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v7
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt -r requirements-dev.txt
      - run: pytest -q
```

Листинг 4 — Пайплайн GitHub Actions (.github/workflows/security.yml)

Для GitLab CI подготовлен файл .gitlab-ci.yml с теми же тремя заданиями и теми же командами. Отличается только синтаксис: в GitLab задания объединяются в стадии (stages) и выполняются в контейнере python:3.12-slim.

```
# Тот же пайплайн для GitLab CI: три независимых задания на одной стадии
image: python:3.12-slim

stages:
  - security

sast:
  stage: security
  script:
    - pip install -r requirements-dev.txt
    - bandit -r app -ll

sca:
  stage: security
  script:
    - pip install -r requirements-dev.txt
    - pip-audit -r requirements.txt

tests:
  stage: security
  script:
```

```
    - pip install -r requirements.txt -r requirements-dev.txt
    - pytest -q
```

Листинг 5 — Пайплайн GitLab CI (.gitlab-ci.yml)

6. Результаты сканирования

6.1. Исходная версия с уязвимостями

Первый коммит репозитория содержит версию сервиса с намеренно оставленными уязвимостями. Bandit нашёл в ней две блокирующие проблемы: хеширование паролей алгоритмом MD5 (уровень High) и SQL-запрос, собранный из строки (уровень Medium).

```
> bandit -r app -ll

Test results:
>> Issue: [B324:hashlib] Use of weak MD5 hash for security. Consider usedforsecurity=False
   Severity: High   Confidence: High
   CWE: CWE-327 (https://cwe.mitre.org/data/definitions/327.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b324_hashlib.html
   Location: app\auth.py:18:11
17    def hash_password(password):
18        return hashlib.md5(password.encode()).hexdigest()
19

--------------------------------------------------
>> Issue: [B608:hardcoded_sql_expressions] Possible SQL injection vector through string-based query construction.
   Severity: Medium   Confidence: Low
   CWE: CWE-89 (https://cwe.mitre.org/data/definitions/89.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b608_hardcoded_sql_expressions.html
   Location: app\database.py:104:17
103    def find_user(username):
104        rows = query(f"SELECT * FROM users WHERE username = '{username}'")
105        return rows[0] if rows else None

--------------------------------------------------

Code scanned:
    Total lines of code: 231
    Total lines skipped (#nosec): 0
    Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
    Total issues (by severity):
        Undefined: 0
        Low: 2
        Medium: 1
        High: 1
    Total issues (by confidence):
        Undefined: 0
        Low: 1
        Medium: 2
        High: 1
Files skipped (0):
```

Рисунок 3 — Результат SAST (Bandit) для исходной версии

pip-audit нашёл уязвимости в двух пакетах: PyJWT 2.3.0 и starlette 0.37.2. Всего записей в отчёте — 32, уникальных идентификаторов уязвимостей — 21

(PyJWT — 14, starlette — 7). Пакет starlette не указан в requirements.txt: это транзитивная зависимость, которую устанавливает fastapi 0.112.0.

```
> pip-audit -r requirements.txt

Found 32 known vulnerabilities in 2 packages
Name      Version ID                  Fix Versions
--------- ------- ------------------- ------------
pyjwt     2.3.0   PYSEC-2022-202      2.4.0
pyjwt     2.3.0   PYSEC-2022-202      2.4.0
pyjwt     2.3.0   PYSEC-2026-120      2.12.0
pyjwt     2.3.0   PYSEC-2026-120      2.12.0
pyjwt     2.3.0   PYSEC-2025-183
pyjwt     2.3.0   PYSEC-2026-179      2.13.0
pyjwt     2.3.0   PYSEC-2026-175      2.13.0
pyjwt     2.3.0   PYSEC-2026-177      2.13.0
pyjwt     2.3.0   PYSEC-2026-177      2.13.0
pyjwt     2.3.0   PYSEC-2026-179      2.13.0
pyjwt     2.3.0   PYSEC-2026-4145     2.14.0
pyjwt     2.3.0   PYSEC-2026-4144     2.14.0
pyjwt     2.3.0   PYSEC-2026-4148     2.14.0
... (ещё 19 строк, полный вывод — в папке reports)
```

Рисунок 4 — Результат SCA (pip-audit) для исходной версии (начало вывода)

Обе команды завершились с кодом 1, поэтому задания SAST и SCA в пайплайне завершаются ошибкой.

6.2. Устранение уязвимостей

Таблица 4 — Найденные уязвимости и исправления

Находка	Было	Стало
B324, High: слабый хеш паролей (app/auth.py)	hashlib.md5(password.encode()).hexdigest()	PBKDF2-HMAC-SHA256, 200 000 итераций, случайная соль для каждого пароля
B608, Medium: SQL-инъекция (app/database.py)	f"SELECT * FROM users WHERE username = '{username}'"	Параметризованный запрос: "... WHERE username = ?", (username,)
B105, Low: секрет подписи JWT в коде (app/auth.py)	SECRET_KEY = "supersecret123"	Секрет из переменной окружения JWT_SECRET; если она не задана — случайный при запуске
SCA: уязвимые версии библиотек (requirements.txt)	fastapi 0.112.0, PyJWT 2.3.0, uvicorn 0.30.0	fastapi 0.162.2, PyJWT 2.15.1, uvicorn 0.54.0 (starlette обновляется до 1.7.0)

Находка B105 имеет уровень Low и не блокирует пайплайн, но секрет в исходном коде позволяет любому, кто видит репозиторий, подделать токен администратора, поэтому она тоже исправлена.

6.3. Повторное сканирование

После исправлений Bandit не находит проблем уровня Medium и High, pip-audit не находит известных уязвимостей, все 8 тестов проходят. Обе команды завершаются с кодом 0.

```
> bandit -r app -ll

Test results:
    No issues identified.

Code scanned:
    Total lines of code: 239
    Total lines skipped (#nosec): 0
    Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
    Total issues (by severity):
        Undefined: 0
        Low: 1
        Medium: 0
        High: 0
    Total issues (by confidence):
        Undefined: 0
        Low: 0
        Medium: 1
        High: 0
Files skipped (0):
```

Рисунок 5 — Результат SAST (Bandit) после исправлений

```
> pip-audit -r requirements.txt

No known vulnerabilities found
```

Рисунок 6 — Результат SCA (pip-audit) после исправлений

```
> pytest -v

tests/test_rbac.py::test_request_without_token_is_rejected PASSED        [ 12%]
tests/test_rbac.py::test_public_is_available_to_every_role PASSED        [ 25%]
tests/test_rbac.py::test_admin_can_create_user PASSED                    [ 37%]
tests/test_rbac.py::test_user_cannot_open_admin_endpoint PASSED          [ 50%]
tests/test_rbac.py::test_moderator_sees_audit_but_not_admin PASSED       [ 62%]
tests/test_rbac.py::test_role_change_applies_without_new_token PASSED    [ 75%]
tests/test_rbac.py::test_user_cannot_promote_himself PASSED              [ 87%]
tests/test_rbac.py::test_user_cannot_read_foreign_profile PASSED         [100%]
======================== 8 passed, 1 warning in 2.10s ========================
```

Рисунок 7 — Тесты ролевой модели после исправлений

В итогах Bandit осталась одна находка уровня Low: строка "bearer" в ответе /login принята за пароль. Это ложное срабатывание: значение — тип токена, а не секрет.

6.4. Запуск в GitHub Actions

Репозиторий размещён на GitHub: https://github.com/is1am/cicd-security-pipeline-v2. Запуск пайплайна для первого коммита завершился ошибкой: задания SAST и SCA не прошли, тесты прошли. Запуск для коммита с исправлениями завершился успешно, прошли все три задания.

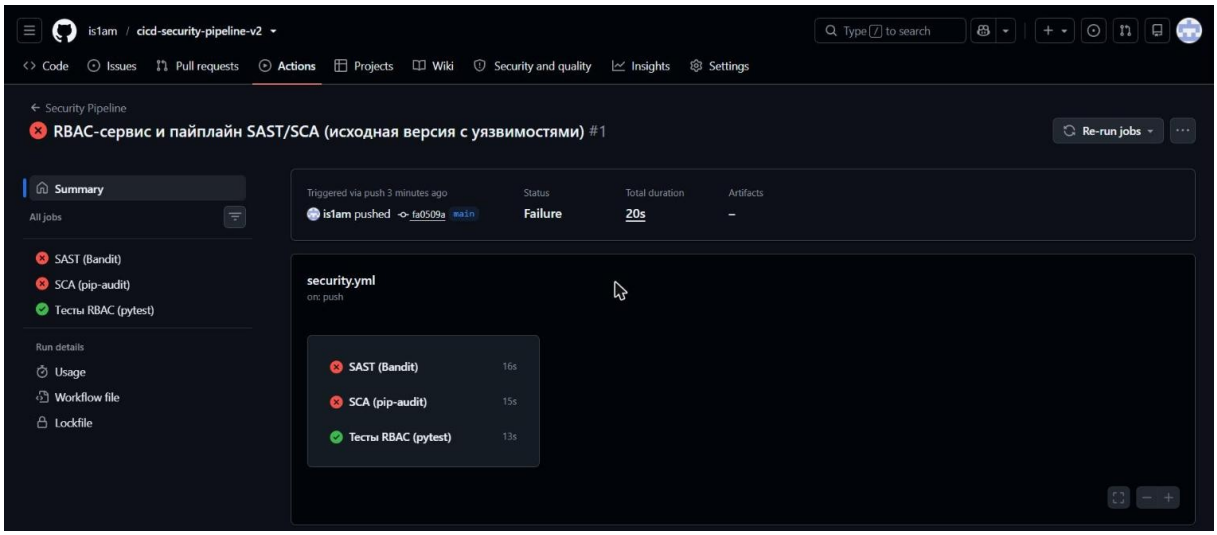


Рисунок 8 — GitHub Actions: запуск для исходной версии

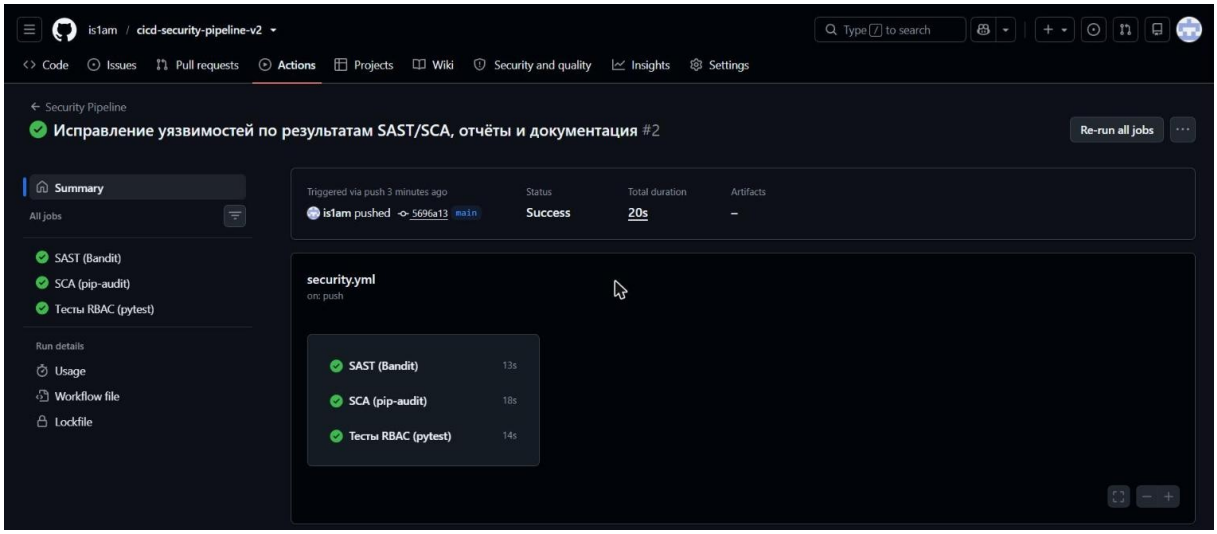


Рисунок 9 — GitHub Actions: запуск после исправлений

7. Тестирование ролевой модели (Negative Testing)

Тесты 1–3 выполнены в Postman: запросы отправлялись сервису, запущенному локально (http://127.0.0.1:8010). Те же запросы собраны в коллекции postman/RBAC_PR5.postman_collection.json и повторяются автоматическими тестами в пайплайне.

7.1. Тест 1 (позитивный)

Администратор входит в систему и получает токен, затем создаёт нового пользователя ivanov с ролью USER. Сервис возвращает 201 Created.

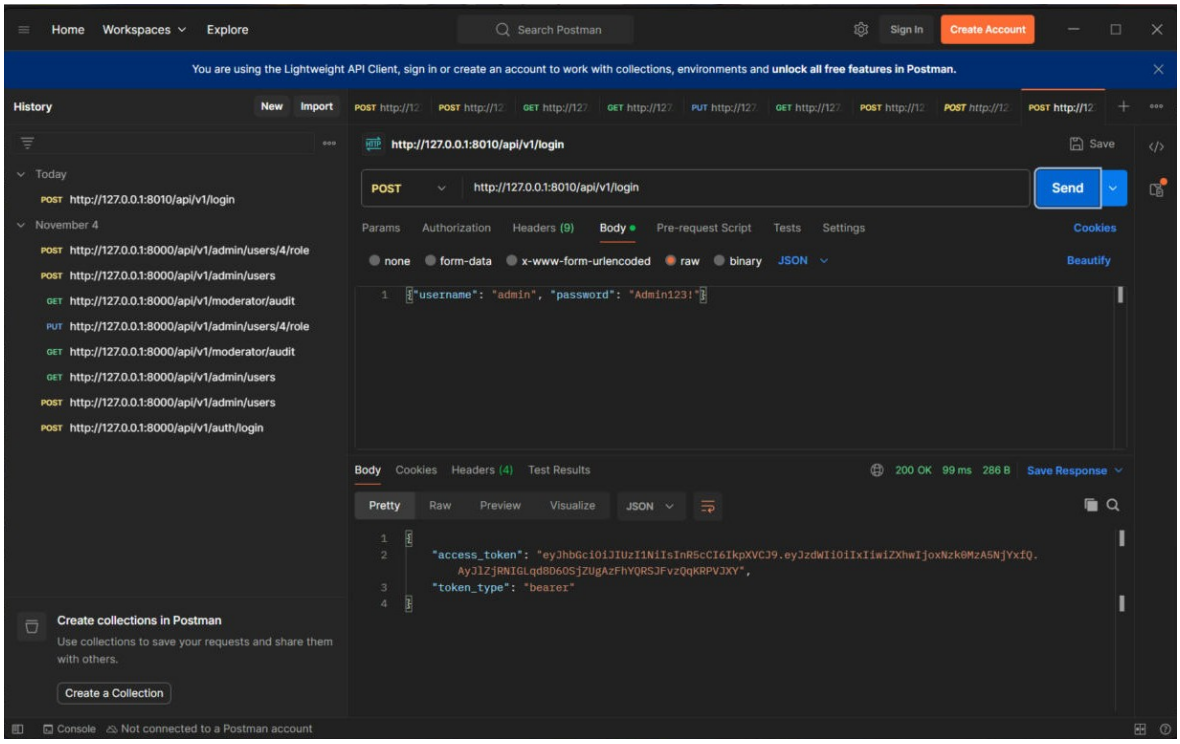


Рисунок 10 — Вход администратора: сервис выдаёт JWT-токен

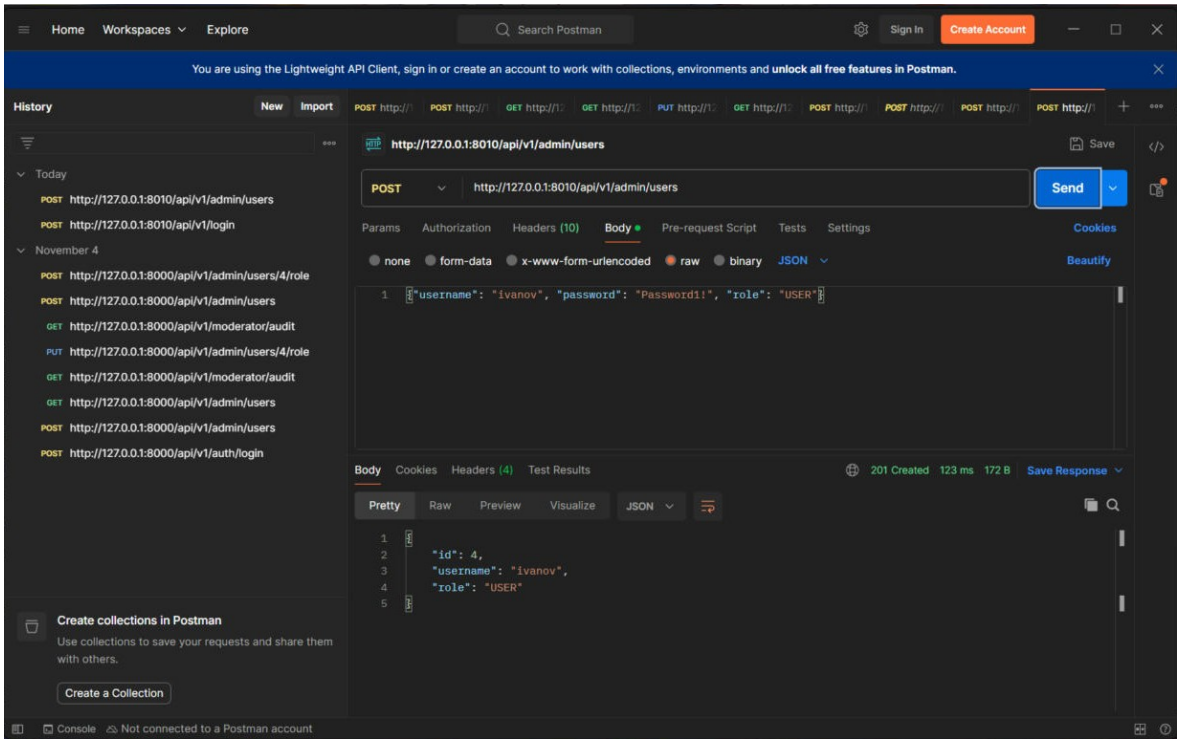


Рисунок 11 — Тест 1: администратор создаёт пользователя, ответ 201 Created

7.2. Тест 2 (негативный)

Рядовой сотрудник (роль USER) обращается к административному эндпоинту /api/v1/admin/users со своим токеном. Запрос отклонён с кодом 403 Forbidden.

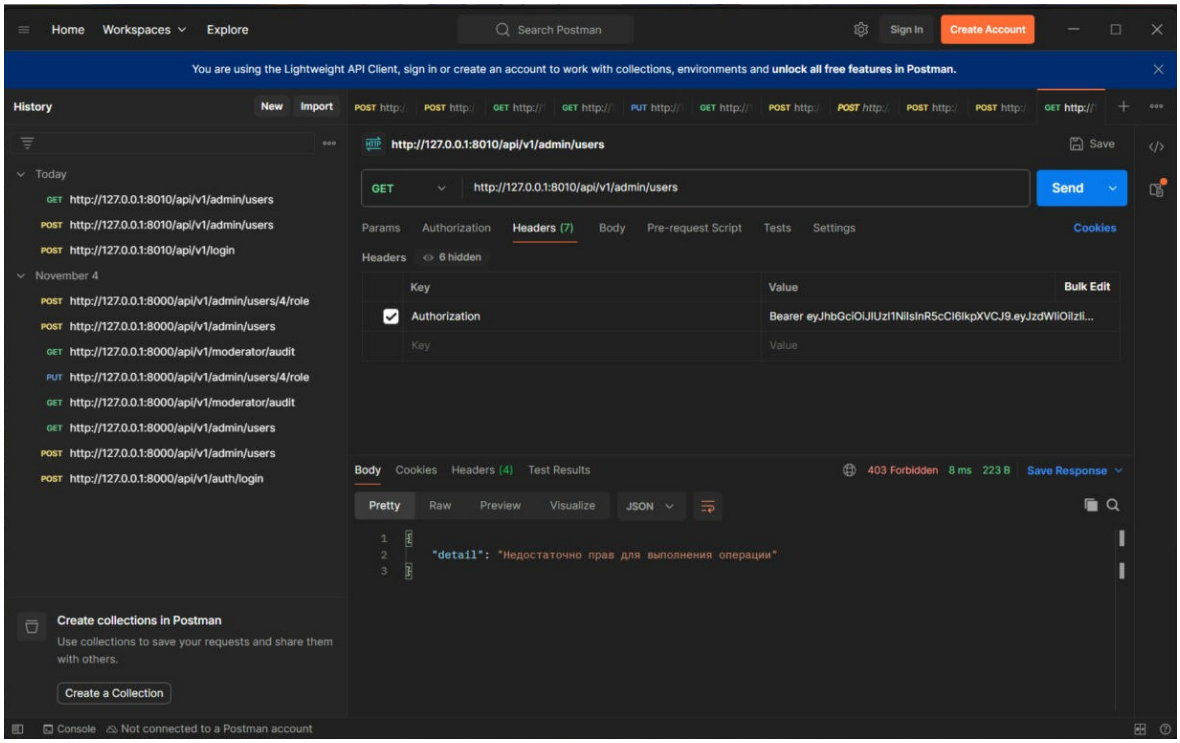


Рисунок 12 — Тест 2: пользователю с ролью USER отказано, ответ 403 Forbidden

7.3. Тест 3 (смена роли «на лету»)

Пользователь получает токен с ролью USER, затем администратор меняет его роль в базе данных. Токен не перевыпускается. Новые права действуют со следующего запроса, задержки нет: в токене хранится только идентификатор пользователя, а роли читаются из БД при каждом запросе. Отзыв роли применяется так же быстро.

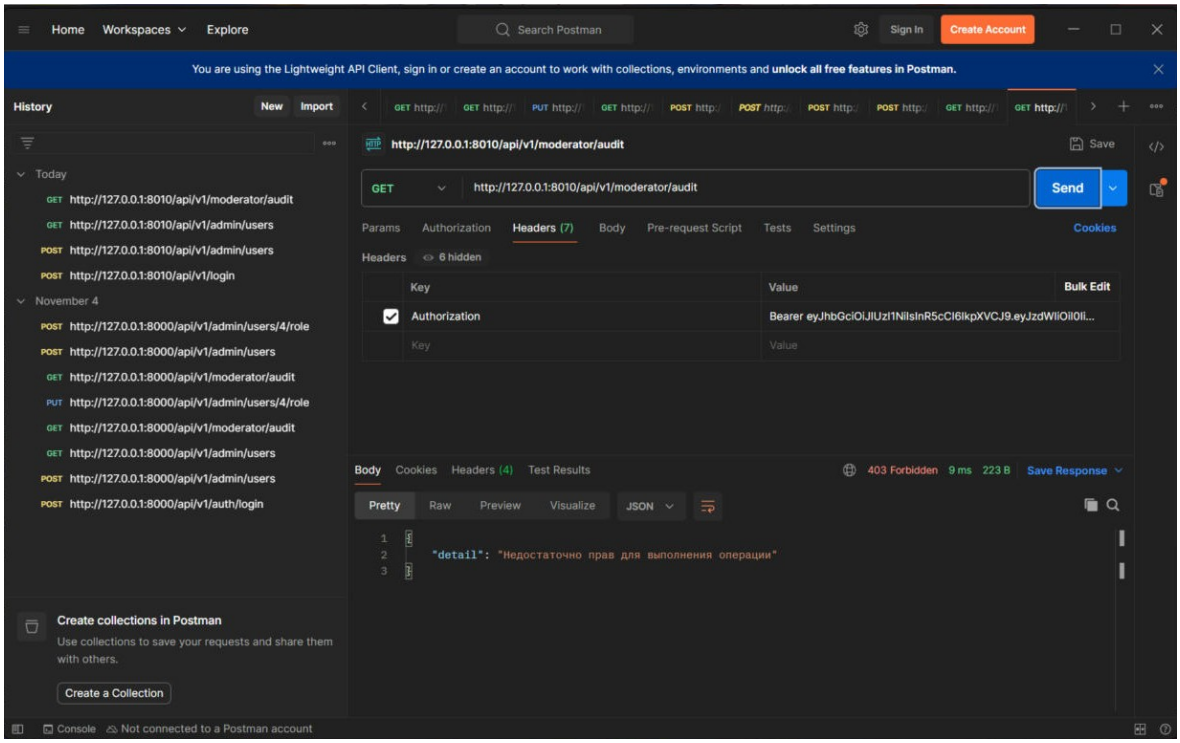


Рисунок 13 — Тест 3, шаг 1: с ролью USER аудит недоступен, ответ 403

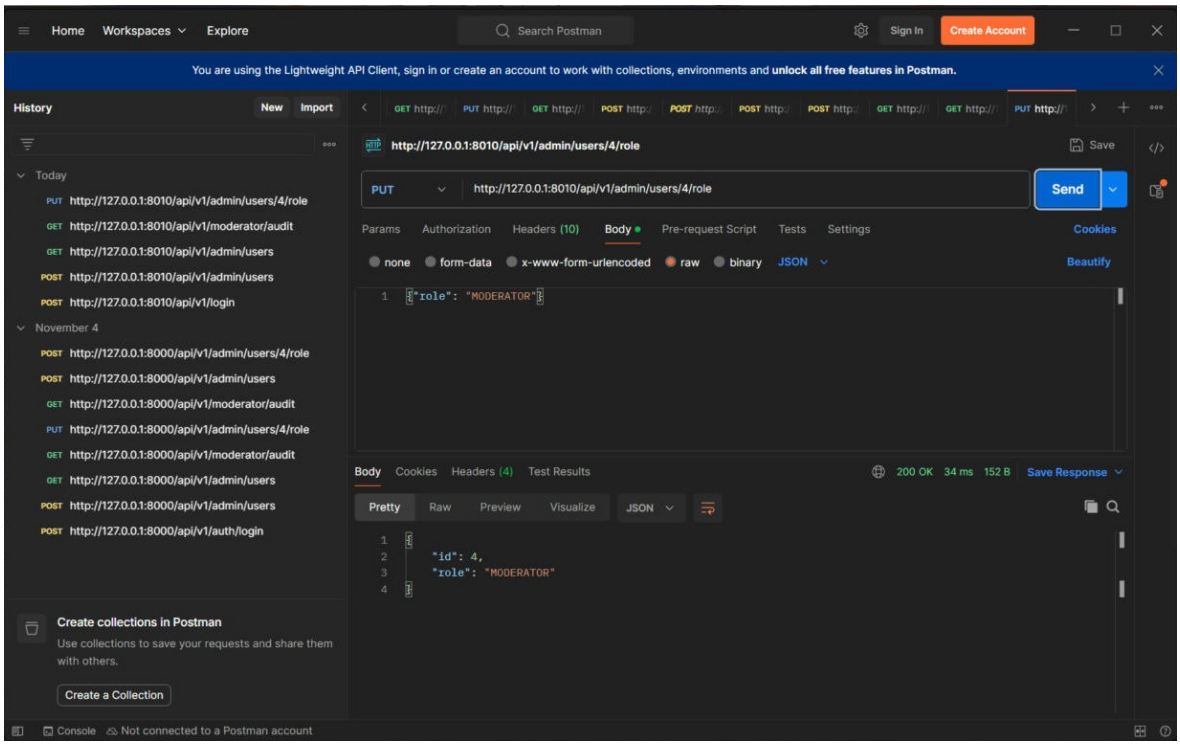


Рисунок 14 — Тест 3, шаг 2: администратор назначает роль MODERATOR

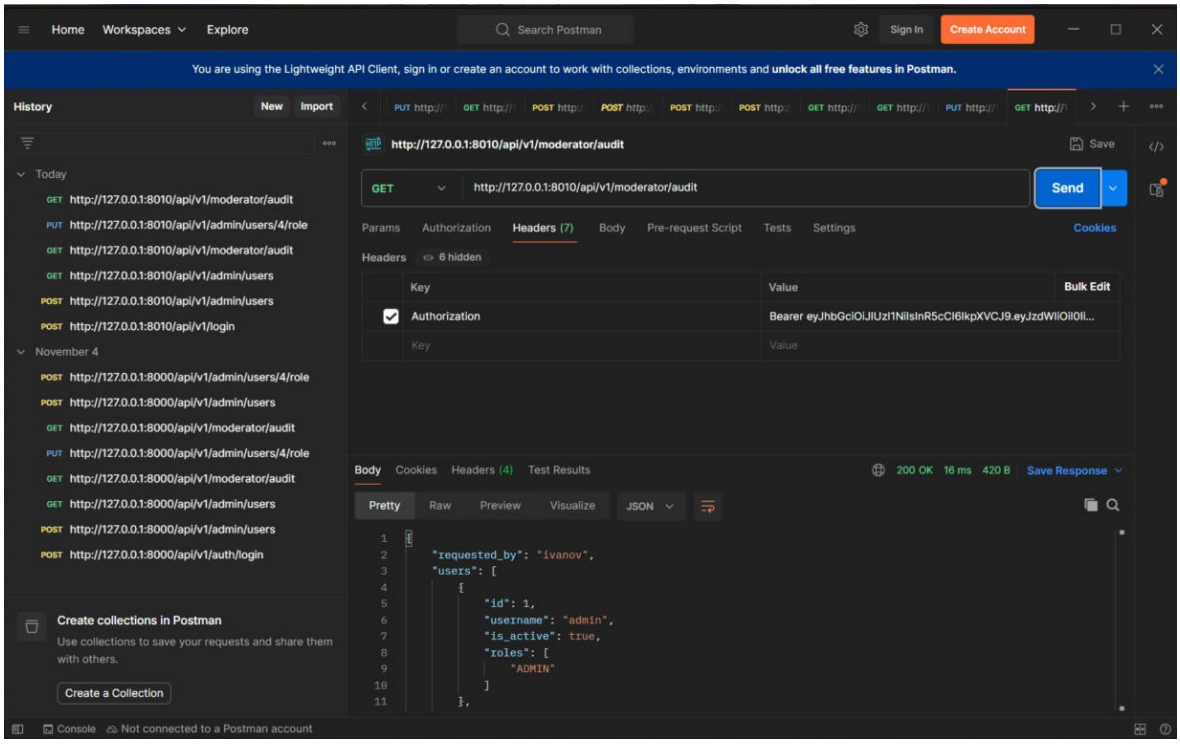


Рисунок 15 — Тест 3, шаг 3: тот же токен, доступ к аудиту появился, ответ 200

Если бы роли записывались в сам токен, изменение роли начинало бы действовать только после истечения срока токена (здесь — до одного часа), и уволенный или пониженный в правах сотрудник сохранял бы доступ всё это время. Цена выбранного решения — дополнительный запрос к БД на каждое обращение.

7.4. Эскалация привилегий и IDOR

Проверены две типовые атаки на неправильно настроенный RBAC: пользователь пытается сам назначить себе роль ADMIN и подставляет в адрес запроса чужой идентификатор. Обе попытки отклонены с кодом 403, собственный профиль пользователю доступен. Запросы выполнены скриптом scripts/api_demo.py, на рисунке — его вывод.

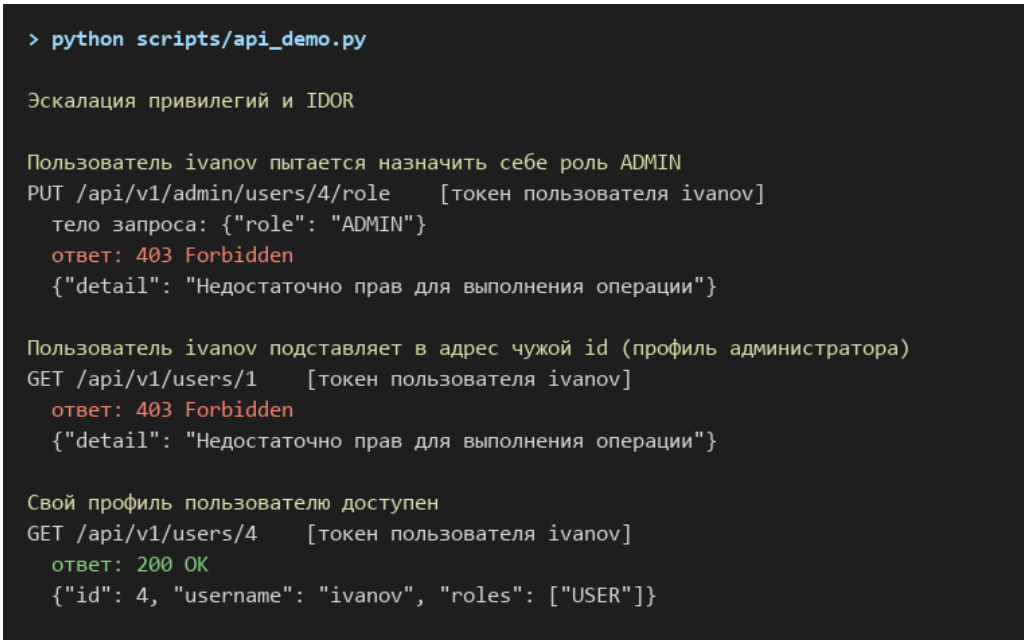


Рисунок 16 — Попытки эскалации привилегий и IDOR

8. Выводы

В работе спроектирована база данных RBAC из пяти таблиц и реализован сервис на FastAPI с тремя уровнями доступа. Негативные тесты показали, что пользователь с ролью USER не может открыть административные эндпоинты, назначить себе роль или прочитать чужой профиль, а смена роли в базе данных действует сразу.

Правильно спроектированный RBAC предотвращает эскалацию привилегий за счёт нескольких решений. Права проверяются на сервере при каждом запросе и не зависят от того, что показывает клиент. Роли и разрешения хранятся в базе данных, а не в токене, поэтому пользователь не может изменить их сам, а отзыв прав действует немедленно. Каждой роли выданы только необходимые разрешения, а менять роли может только обладатель права MANAGE_USERS. Доступ к объекту проверяется по идентификатору из токена, а не из адреса запроса, что закрывает IDOR.

Пайплайн CI/CD дополняет ролевую модель: он проверяет каждое изменение кода. В исходной версии Bandit нашёл слабый хеш паролей и SQL-инъекцию, pip-audit — уязвимости в двух библиотеках (уникальных идентификаторов — 21). Обе проблемы в коде позволяли обойти ролевую модель: через SQL-инъекцию или подбор MD5-хешей можно получить учётную запись администратора при корректно настроенных ролях. После исправлений

пайплайн проходит полностью. Тесты ролевой модели в пайплайне не дают случайно ослабить проверку прав при дальнейших изменениях.